

sudTP 2 - Introduction à Flask

But du TP

Le but du TP est de mettre en place un serveur BackEnd, avec différentes routes, qui seront sécurisées.

Les routes devront vous permettre d'afficher du contenu.

Vous devrez également mettre en place un token afin de garantir une connexion Sécurisée.

Dans ce TP vous ne prendrez pas en compte les failles XSS et injections SQL.

Pour les bouts de code je vous laisse aller voir le cours, tout est dedans !

Attention, si vous êtes sur Linux dans l'invité de commande ne vous connectez pas en tant que "root" sinon vous risquez d'avoir des soucis de droits. But du TP Mise en place et configuration du serveur Création du serveur Configuration du serveur Création des premières routes Route about Route hello API et Database Installation package db, importation et configuration Création de notre d'Afficher les utilisateurs Afficher un utilisateur Insertion d'un utilisateur Sécurisation d'une API Configuration du package Sécuriser l'API Data du token Partie Bonus

TP 2 - Introduction à Flask 2 Avant toute chose, sur le réseau de l'IUT vous risquez d'avoir quelques soucis liés au

proxy, voici donc les 2 commandes à exécuter avant de commencer le TP :

important

```
export http_proxy=cache-etu.univ-artois.fr:3128
export https_proxy=cache-etu.univ-artois.fr:3128
```

Mise en place et configuration du serveur

Installation de Python et Flask

```
sudo apt install python3 python3-pip
```

Ensuite il faut installer Flask, voici la commande à utiliser :

```
pip install Flask
```

La commande pour voir la version de Flask

```
root@rt-mv:/home/administrateur# ^C
root@rt-mv:/home/administrateur# python3 -m flask --version
Python 3.10.12
Flask 3.1.0
Werkzeug 3.1.3
root@rt-mv:/home/administrateur#
```

Création du serveur

Dans un premier temps vous allez créer un dossier sur votre PC, c'est ce dossier qui va contenir notre petit projet.

Ensuite nous allons créer notre fichier nommé "app.py", puis vous allez venir créer la base de votre serveur.

Je veux que le serveur tourne sur 127.0.0.1 et sur le port 5000.

N'oubliez pas de configurer vos variables d'environnement, sinon vous aurez une erreur et mettez en mode "développement" sinon vous travaillez en mode production.

Écriture du code de base du serveur Flask

Nano app.py

Puis on ajout le code

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def home():
```

```
    return "Bienvenue sur mon serveur Flask !"
```

```
if __name__ == '__main__':
```

```
    app.run(host='127.0.0.1', port=5000, debug=True)
```

Enregistrez avec CTRL+X, puis Y, et Entrée.

4. Configuration des variables d'environnement

sous linux

```
export FLASK_APP=votre_app.py
```

```
export FLASK_ENV=development
```

5- Lancer le serveur Flask

flask run

Ou

```
administrateur@rt-mv:~$ export FLASK_APP=app.py
administrateur@rt-mv:~$ python3 -m flask run
Usage: python -m flask run [OPTIONS]
Try 'python -m flask run --help' for help.

Error: Could not import 'app'.
administrateur@rt-mv:~$ python3 -m flask run
Usage: python -m flask run [OPTIONS]
Try 'python -m flask run --help' for help.

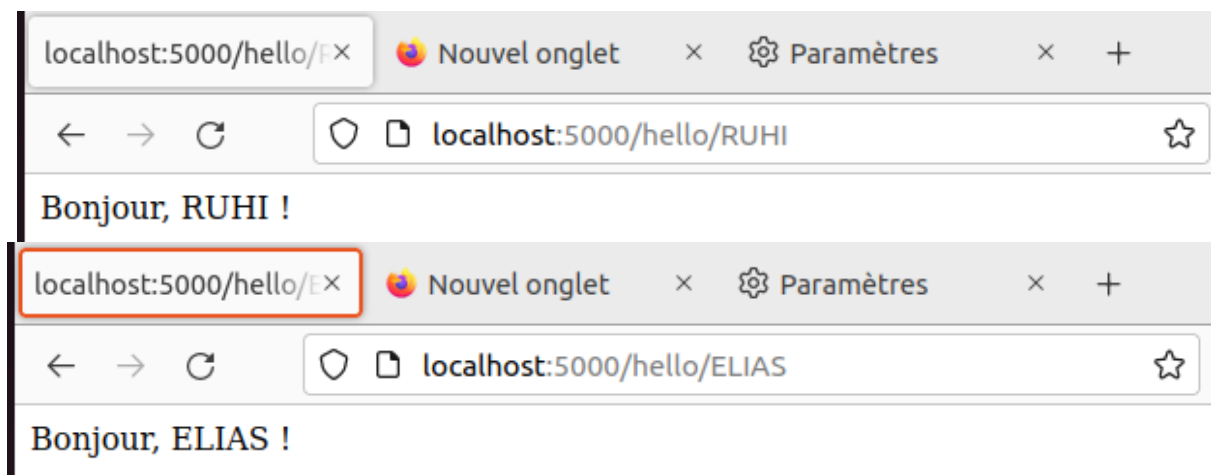
Error: Could not import 'app'.
administrateur@rt-mv:~$ ls
app.py  Documents  Modèles  Musique  rubi  Téléchargements  Vidéos
```

Python3 -m flask run

Configuration du serveur

```
Ouvrir  [icon] app.py ~/app.py Enregistrer [menu] [minus]

1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "Le serveur fonctionne correctement !"
8
9 @app.route('/about')
10 def about():
11     return "Bonjour, je suis Ruhi Sherzad"
12
13 @app.route('/hello/<name>')
14 def hello(name):
15     return f"Bonjour, {name} !"
16
17 if __name__ == '__main__':
18     app.run(debug=True)
19
```



API et Database

Installation d'un serveur mysql et configuration

Il faut tout d'abord installer un serveur mysql :

```
sudo apt install mysql-server
```

Ensuite il faut finir la configuration de l'installation :

```
sudo mysql_secure_installation
```

Après avoir réaliser l'installation il faut venir modifier le mot de passe utilisateur :

```
sudo mysql
```

Une fois dans le shell mysql, vous devez lancer la requête SQL suivante (en modifiant le mot de passe) :

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'password';
```

```
Remove test database and access to it? (Press y|Y for Yes, any other key for No) : n
... skipping.
Reloading the privilege tables will ensure that all changes made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
administrateur@rt-mv:~/app.py$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.41-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
-> 'password'
->
->
```

MDP pour mysql⇒password

Acces msq⇒ mysql -u root -p

Requete sql⇒ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'password';

Installation package db, importation et configuration

INFO

- Il est possible que les changements ne soient pas pris en compte tout de suite, si c'est le

cas alors nous allons lancer la commande pour "refresh" les privilèges :

SUCCESS

Désormais pour vous connecter à votre serveur mysql il faudra utiliser la commande suivante :

- Installation package db, importation et configuration

Après avoir créer des routes, maintenant passons à l'étape suivante : MySQL

Vous disposez d'un accès à une DB, si ce n'est pas le cas installez un logiciel permettant d'avoir accès à une db.

Pour installer le package il vous faudra sûrement rajouter et/ou modifier le proxy, pour cela vous devez utiliser une commande que vous avez utilisé dans un autre TP.

Installez le package Python pour MySQL :

Installez le package Python pour MySQL :

```
pip install mysql-connector-python  
  
# si soucis de version  
pip3 install mysql-connector-python
```

Puis importez le dans votre application :

```
import mysql.connector as MC
```

Ensuite je vous laisse chercher dans le cours pour créer la connexion à votre db. Attention celle-ci devra être dans un **try except**.

```
administrateur@rt-mv:~/app.py$ pip install mysql-connector-python  
Defaulting to user installation because normal site-packages is not writeable  
Collecting mysql-connector-python  
  Downloading mysql_connector_python-9.2.0-cp310-cp310-manylinux_2_28_x86_64.whl (33.9 MB)  
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 33.9/33.9 MB 1.1 MB/s eta 0:00:00  
Installing collected packages: mysql-connector-python  
Successfully installed mysql-connector-python-9.2.0  
administrateur@rt-mv:~/app.py$
```

```

administrateur@rt-mv:~/app.py$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 8.0.41-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> pip install mysql-connector-python
-> import mysql.connector as MC
-> ^C
mysql> exit
Bye
administrateur@rt-mv:~/app.py$

```

Ensuite je vous laisse chercher dans le cours pour créer la connexion à votre db. Attention

celle-ci devra être dans un try except.

On rajoute la ligne **import mysql.connector as MC**

```

from flask import Flask
import mysql.connector as MC
app = Flask(__name__)

@app.route('/')
def home():
    return "Le serveur fonctionne correctement !"

@app.route('/about')
def about():
    return "Bonjour, je suis Ruhi Sherzad"

@app.route('/hello/<name>')
def hello(name):
    return f"Bonjour, {name} !"

if __name__ == '__main__':
    app.run(debug=True)

```

Création de notre db

- Vous allez devoir créer une db et une table nommée “users” avec cette structure :
- id : int auto_increment primary key
- pseudo : varchar(25)
- password : varchar(255)

Si vous souhaitez vous connectez à mysql via un invité de commande voici la commande

Donc soit vous le faites à la main soit vous téléchargez le fichier “users.sql” disponible dans moodle à la section du TP 2.

Création de la database et la table

Avant toute chose il faut créer la database (où sera stockée la table) :

```
CREATE DATABASE db_tp2;
```

Ensuite il faut exécuter le fichier SQL pour créer la table :

```
mysql -u root -p db_tp2 < users.sql
```

Et ensuite il faut vérifier si tout est bon :

```
USE db_tp2;  
SHOW TABLES;
```

Vous pouvez même regarder si votre table 'users' contient bien les données :

```
SELECT * FROM users;
```

```
administrateur@rt-mv:~/app.py$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.0.41-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT * FROM user;
ERROR 1046 (3D000): No database selected
mysql> show databases;
+-----+
| Database |
+-----+
| db_tp2   |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0,00 sec)

mysql> exit
Bye
```

```
administrateur@rt-mv:~/app.py$ mysql -u root -p db_tp2 < users.sql
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 32
Server version: 8.0.41-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE db_tp2;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_db_tp2 |
+-----+
| users             |
+-----+
1 row in set (0,01 sec)

mysql> SELECT * FROM users;
+-----+
| id | pseudo | password |
+-----+
| 1  | user1  | 1234     |
| 2  | user2  | 4321     |
+-----+
2 rows in set (0,00 sec)

mysql>
```

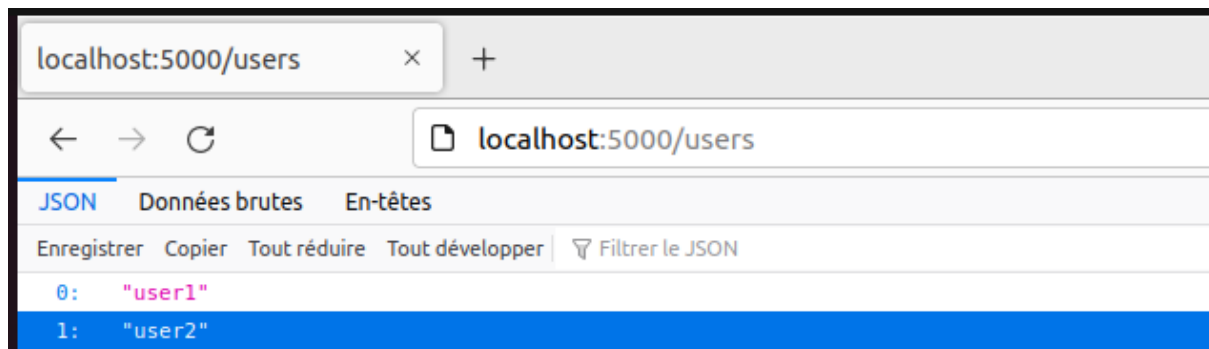
Afficher les utilisateurs

En utilisant le cours sur Flask, créez une route nommée “users”, cette route devra afficher la liste des utilisateurs (uniquement le pseudo), l’affichage devra être au format JSON. Vous allez donc devoir utiliser un logiciel permettant de réaliser des requêtes API REST (Insomnia, Postman, etc...).

En utilisant cette code

```
1 import sys
2
3
4 app = Flask(__name__)
5
6 @app.route("/about")
7 def about():
8     return "Bonjour, je suis Benjamin"
9
10 @app.route("/hello/<string:prenom>")
11 def hello(prenom):
12     return f"Bonjour à toi {prenom} mon ami(e) !"
13
14 def get_db_connection():
15     """Créer une nouvelle connexion à la base de données."""
16     return MC.connect(
17         host="localhost",
18         user="root",
19         password="password",
20         database="db_tp2"
21     )
22
23 @app.route("/users")
24 def get_users():
25     conn = get_db_connection()
26     cursor = conn.cursor(dictionary=True)
27     cursor.execute("SELECT pseudo FROM users")
28     users = cursor.fetchall()
29
30     # Fermer le curseur et la connexion
31     cursor.close()
32     conn.close()
33
34     # Retourner uniquement la liste des pseudos
35     return jsonify([user["pseudo"] for user in users])
36
37
38 if __name__ == "__main__":
39     app.run(host="127.0.0.1", port=5000, debug=True)
```

On a réussi à afficher notre liste sur la page internet:



Afficher un utilisateur

Objectif suivant : Créer une route permettant d’afficher l’ID et le pseudo d’un utilisateur en utilisant son pseudo comme paramètre, si aucun résultat alors j’affiche un message d’erreur. Exemple : `http://localhost:5000/user/maxime` → not found
Exemple : `http://localhost:5000/user/user1` → id : 1, pseudo : user1
Résultat au format JSON et nom de la route “user”

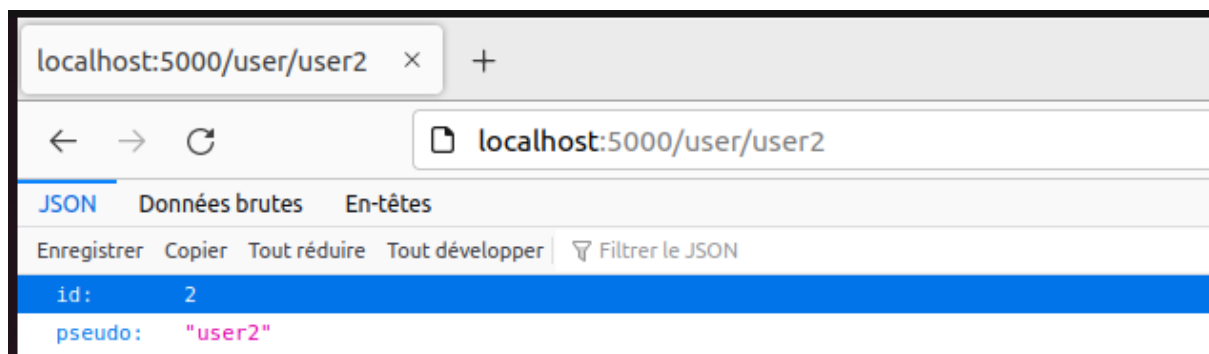
Le code

```
from flask import Flask, jsonify
import mysql.connector as MC
app = Flask(__name__)

db = MC.connect(
    host="localhost",
    user="root",
    password="password",
    database="db_tp2"
)

@app.route("/user/<string:pseudo>")
def get_user(pseudo):
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT id, pseudo FROM users WHERE pseudo = %s", (pseudo,))
    user = cursor.fetchone()
    if user:
        return jsonify(user)
    else:
        return jsonify({"error": "Utilisateur non trouvé"}), 404

if __name__ == "__main__":
    app.run(debug=True)
résultat: pour l'affiche python3 -m flask run
```



Insertion d'un utilisateur

En utilisant le cours sur Flask et sur le SQL, votre objectif est le suivant :

Créer une route permettant de créer un utilisateur dans la table "users", le pseudo et le mot de passe devront être des paramètres de l'api.

Les 2 paramètres devront être obligatoires, si ce n'est pas le cas alors retournez un message d'erreur.

Conseils :

Le type de la requête sera POST et non du GET.

SQL : utiliser INSERT INTO

Une fois cela fait, merci d'appeler le professeur afin de valider le travail effectué.

Route sécurisée avec token

Le code

```
app.py × App.py
1 from flask import Flask, request, jsonify
2 import jwt
3
4 app = Flask(__name__)
5 app.config["SECRET_KEY"] = "secret"
6
7 def token_required(f):
8     def decorated(*args, **kwargs):
9         token = request.headers.get("x-access-token")
10        if not token:
11            return jsonify({"error": "Token manquant"}), 403
12        try:
13            jwt.decode(token, app.config["SECRET_KEY"], algorithms=["HS256"])
14        except:
15            return jsonify({"error": "Token invalide"}), 403
16        return f(*args, **kwargs)
17    return decorated
18
19 @app.route("/users", methods=["GET"])
20 @token_required
21 def get_users():
22     return jsonify({"message": "Accès autorisé, voici la liste des utilisateurs"})
23
24 if __name__ == "__main__":
25     app.run(debug=True)
26
```

Sécurisation d'une API

Configuration du package

Maintenant vous devez mettre en place une route permettant de générer un token.

Installation du package JWT :